

GATEWEB · FINAL 12 HOURS

面試前 12 小時整合版

關網資訊 AI 應用工程師：今晚到面試前，只讀這一份

依據：面試趣 7 篇關網心得全文（含同職缺錄取者）、104 職缺頁、你的履歷與真實分工

【今晚填】的地方只有你知道真實情況，睡前寫好，不要現場編

時間表：扣掉睡覺，你只有 6 小時

睡前 3 小時

時間	做什麼	用哪份
0:00–0:30	讀 PART 2 筆試衝刺，讀一次就好，不要抄	本份
0:30–1:00	模擬卷 A 限時 30 分鐘，手機放遠（做過 A 就改做 B）	關網筆試題庫 PDF
1:00–1:20	對答案，錯的題目只看解析，不要回頭讀講義	同上
1:20–2:30	PART 3 面談：先把【今晚填】寫好，每題小聲講 2 次	本份
2:30–2:50	PART 4 技術線自查：遮住右邊兩欄，自己講出一句	本份
2:50–3:00	照 PART 6 準備衣服、履歷、水、路線，然後睡覺	本份

起床後約 3 小時（含通勤）

時間	做什麼
起床後 30 分鐘	梳洗、吃早餐、喝溫水。不要讀書
接下來 20 分鐘	小聲講一次：自我介紹、為什麼選關網、解釋 AI
出門前 5 分鐘	打開關網官網的電子發票查詢頁實際看一次
通勤路上	只讀 PART 2 最後的易混淆對照、PART 5 公司一頁
到場前 10 分鐘	喝水、慢慢深呼吸、手機靜音

不要做的事

不要	原因
讀 LLM 講義 CH01–CH10 的底層	筆試考 Agent、RAG、前後端、測試安全，不考數學
做隨堂測驗、改簡報	心得沒有人要求簡報；新題目只會增加焦慮
熬夜刷題	你要撐 1.5–2 小時對談，睡眠比多 10 題更重要，聲音也需要休息

明天會發生什麼（依 7 篇心得）

階段	內容	你要注意的
報到	可能先填基本資料或問卷（後端 2025.03）	帶筆，字寫工整
筆試	30 題全選擇、30 分鐘、不能用手機；考 AI Agent、RAG、前後端調用、測試安全	一題 1 分鐘，不會的先跳過
HR	履歷、為什麼投遞、把 HR 當不懂的客戶，解釋什麼是 AI	不要用專有名詞
主管 + 團隊	履歷與專案追問、大量情境題（衝突、意見不合、主管特質、優缺點、三個條件）	同一題兩關答案要一致
結束	最後問你有沒有問題；三天內通知	準備 2 個反問

七篇心得的共同點

1. 專案一定會被問，而且會追問：技術題從作品或筆試題延伸、最大困難怎麼解決、你在團隊擔任什麼角色。
2. 自我介紹多篇都有：HR 說不一定要，但準備一個 50 秒版本。
3. 衝突題不能只答「沒有」：有一篇這樣答，被連續追問參與度、怎麼幫忙、有沒有成效，最後被判定沒參與。
4. 會要你向外行人解釋：你的職缺是 AI，Java 職缺是 API。
5. 可能被咄咄逼人：兩篇說主管或營運長語氣很衝，作者認為是在測抗壓。
6. 提問快、喜歡好相處不急躁的人：先想一秒再答，答短一點。

你的現況

- 104 顯示應徵人數「30 人以上」，那是按下應徵的人數，不是面試人數。你是被篩進來的。
- 需求 1 人。題目不難，比的是講得具體、穩定、誠實。

筆試四大範圍：每行一個會考的觀念

A. AI Agent、Tool Calling、MCP

觀念	記住這句
Agent 與 Workflow	Workflow 的步驟由程式寫死； Agent 由模型決定下一步（呼叫哪個工具、要不要繼續）
Agent 的組成	LLM + 工具 + 規劃（Planning）+ 記憶（Memory）
ReAct	Reasoning + Acting：思考 → 行動（呼叫工具）→ 觀察結果，重複到完成
Tool / Function Calling	模型只 提出 要呼叫的工具與參數（JSON）， 真正執行的是你的程式 ，結果再回傳給模型
工具定義	用 JSON Schema 描述名稱、說明、參數；說明寫得清楚，模型才選得對
MCP	Model Context Protocol，Anthropic 2024 提出的 開放協定 ，標準化 AI 應用連接工具與資料的方式
MCP 架構	Host（AI 應用）→ Client → Server ；Server 提供 Tools、Resources、Prompts
MCP 通訊	訊息格式 JSON-RPC 2.0 ；傳輸： stdio （本機）、 Streamable HTTP （遠端）
MCP 的價值	把「N 個應用 × M 個工具」各自串接，變成 N + M
Function Calling 與 MCP	前者是 模型 API 的能力 ；MCP 是 工具怎麼接進來的標準 ，兩者互補，不是二選一
Planning	把大任務拆成小步驟（Plan-and-Execute）；Reflection 是讓模型檢查自己的結果
Memory	短期：這次對話的內容與狀態（在 context window 內）；長期：存到外部資料庫，需要時再檢索
Multi-Agent	多個角色分工（例如一個負責分派、其他負責執行）；代價是成本、延遲、除錯變難
LangGraph	State （共享狀態）、 Node （做事的步驟）、 Edge （下一步）、 Conditional Edge （依條件分支）、 Checkpoint （保存狀態，可中斷、恢復、等人工確認）
防止 Agent 失控	最大迭代次數、逾時、工具白名單、 寫入或刪除等高風險動作需人工確認 、記錄每一步軌跡（LangSmith、Langfuse）
Temperature	越低輸出越穩定；工具呼叫、資料抽取用低溫

B. RAG

觀念	記住這句
RAG 是什麼	Retrieval-Augmented Generation：先檢索資料，再放進 prompt 讓模型依據回答
解決什麼	知識過時、公司私有資料、降低幻覺； 不用重新訓練模型
離線流程	文件解析 → 清洗 → 切塊（chunk） → embedding → 存入向量資料庫（連同 metadata）

觀念	記住這句
線上流程	問題轉 embedding → 相似度搜尋取 top-k → (rerank) → 組 prompt → 生成並附出處
Embedding	把文字轉成向量，語意越接近，向量越接近；常用 cosine similarity (看夾角)
同一個模型	查詢和文件必須用同一個 embedding 模型；換模型要重建索引
Chunk 大小	太大：雜訊多、重點被稀釋；太小：缺上下文。overlap (重疊) 避免語意被切斷
Hybrid Search	BM25 關鍵字 (精準比對統編、法條編號、專有名詞) + 向量 (語意相近)
Rerank	用 cross-encoder 對候選重新排序，準但慢，所以只對 top-k 候選做
向量索引	ANN (近似最近鄰) 用一點準確度換速度；HNSW 是圖結構，快但吃記憶體；IVF 先分群再找
向量資料庫	Qdrant、Chroma、Milvus、pgvector (PostgreSQL 的擴充)
評估	檢索端：Recall@k、Hit Rate、MRR；生成端：faithfulness (有沒有忠於資料)、answer relevance
答錯怎麼查	先分辨是檢索沒撈到，還是撈到了但生成錯。前者調切塊、檢索參數、加 hybrid；後者調 prompt 或模型
權限控管	用 metadata 在檢索階段就過濾使用者看不到的文件，不能靠 prompt 叫模型不要說
RAG 與微調	知識常變動 → RAG；要改語氣、格式、特定任務行為 → 微調 (例如 LoRA)

C. 前後端調用

觀念	記住這句
HTTP 方法	GET 讀取；POST 新增 (非冪等)；PUT 整筆取代；PATCH 部分更新；DELETE 刪除
冪等	同一個請求做 1 次和做 N 次結果一樣：GET、PUT、DELETE 是；POST 不是
2xx	200 成功；201 已建立；204 成功但沒有內容
3xx	301 永久轉址；302 暫時轉址；304 沒變動、用快取
4xx	400 請求格式錯；401 沒認證 (沒登入、token 無效)；403 有認證但沒權限；404 找不到；405 方法不允許；409 衝突；422 參數驗證失敗 (FastAPI 常見)；429 請求太多 (限流)
5xx	500 伺服器錯誤；502 閘道收到錯誤回應；503 服務暫時無法使用；504 閘道逾時
REST	URL 用名詞表示資源 (/invoices/123)，動作用 HTTP 方法；無狀態
CORS	瀏覽器的同源政策：協定、網域、port 任一不同就是跨域，伺服器要回 Access-Control-Allow-Origin；複雜請求會先送 OPTIONS 預檢
CORS 的本質	是瀏覽器的限制，不是伺服器的安全防護，curl 或後端呼叫不受影響
Session 與 JWT	Session：狀態存在伺服器；JWT：伺服器簽章的 token，Header.Payload.Signature

觀念	記住這句
JWT 重點	Payload 只是 Base64URL 編碼 ，沒有加密，不能放機密；簽章只防竄改；很難提前作廢 → 短效期 + refresh token
帶 token	Header : Authorization: Bearer <token >
串流	SSE ：伺服器單向推送，基於 HTTP， LLM 逐字輸出常用 ； WebSocket ：雙向即時
FastAPI 非同步	async def 裡 不能呼叫會阻塞的 I/O ，會卡住整個事件迴圈；同步函式用 def，FastAPI 會丟到 threadpool
重試	逾時與 429 用 指數退避 重試；會改資料的操作要能冪等（idempotency key），避免重複開立
LLM API 異常	429 限流、逾時、超過 context 上限、輸出格式不符 → 驗證格式、失敗重試、備援模型

D. 測試與安全

觀念	記住這句
測試金字塔	單元測試最多最快 → 整合測試 → E2E 最少最慢
Mock	測試時用假的替換外部依賴（例如 LLM API），讓測試 穩定、快、不花錢
LLM 輸出測試	不逐字比對 ；驗證 JSON 結構、關鍵事實、禁止內容；固定測試集做回歸；LLM 當評審要人工校準
壓力測試指標	RPS / QPS、錯誤率、延遲看 P95、P99 ，不是平均；工具 Locust、JMeter、k6
編碼、加密、雜湊	編碼（Base64） 可還原、沒有金鑰 ；加密 可還原、要金鑰 （AES 對稱、RSA 非對稱）；雜湊 不可還原 （SHA-256）
存密碼	用 bcrypt、Argon2 這類加鹽的慢雜湊，不是加密
SQL Injection	用 參數化查詢 或 ORM，不要字串拼接 SQL
XSS	輸出前做編碼、設定 CSP、避免 innerHTML 塞使用者輸入
CSRF	SameSite Cookie、CSRF token
IDOR	改網址的 id 就看到別人資料 → 伺服器端檢查資源是不是這個人的
OWASP Top 10 (2021)	第一名 Broken Access Control （存取控制失效）
OWASP LLM Top 10 (2025)	LLM01 Prompt Injection 、LLM02 敏感資訊洩漏、LLM03 供應鏈、LLM04 資料與模型投毒、LLM05 不當輸出處理 、LLM06 過度代理（Excessive Agency） 、LLM07 System Prompt 洩漏、LLM08 向量與 Embedding 弱點、LLM09 錯誤資訊、LLM10 無上限資源消耗
Prompt Injection	直接：使用者輸入惡意指令； 間接：指令藏在網頁、文件、email 裡被 RAG 或 Agent 讀進來
怎麼防	不能只靠 system prompt ：最小權限、工具白名單、高風險動作人工確認、外部內容當資料不當指令、輸出驗證
API Key	放後端或環境變數， 不放前端、不進 Git

觀念	記住這句
Rate limit	防濫用、防成本失控

E. LLM 基本 (少量出現)

觀念	記住這句
Token	模型處理文字的單位；計費和 context 上限都用 token 算
Context window	一次能處理的 token 上限 (輸入 + 輸出)
幻覺	模型自信地講錯 → 用 RAG、要求附出處、程式驗證
Prompt 技巧	清楚指令、給角色、few-shot 範例、要求結構化輸出
地端模型	Ollama 在本機跑開源模型 (Llama、Qwen)，資料不出公司

易混淆對照 (通勤時只看這張)

容易搞混	差別
401 與 403	401 你是誰我不知道；403 我知道你是誰，但你不能做
編碼、加密、雜湊	Base64 誰都能還原；加密要金鑰才能還原；雜湊不能還原
JWT	簽章防竄改， 沒有加密 ，Payload 看得到
CORS	瀏覽器擋的，不是伺服器的防護
Workflow 與 Agent	程式決定步驟 vs 模型決定步驟
Function Calling 與 MCP	模型能力 vs 接工具的標準協定
模型執行工具？	不會 ，模型只提出，程式執行
RAG 與微調	換知識用 RAG；換行為用微調
BM25 與向量	字面精準 vs 語意相近；兩個一起用叫 Hybrid
Rerank 為什麼只排 top-k	準但慢
SSE 與 WebSocket	單向推送 vs 雙向即時
平均與 P95	平均會掩蓋少數很慢的請求，看 P95、P99
權限控管放哪	檢索或後端層過濾，不靠 prompt

答題技巧

- **30 題 30 分鐘**：先把有把握的做完，不確定的做記號，最後回頭。
- 選項出現「**一定、完全、只要...就能保證**」通常是錯的。
- 安全類情境題，選「**最小權限、伺服器端驗證、人工確認、記錄稽核**」的方向。
- 考完把**沒把握的題目記在腦中**，面談可能從考題延伸。

面談：一定要能開口講的題目

原則：每題 30–60 秒就停，讓對方追問。答短符合他們提問快的節奏，也最省聲音。不要背稿，記住順序用自己的話講。

1. 自我介紹（約 50 秒）

面試官好，我是邱冠文，今年從明新科技大學資訊工程系畢業。

我之前讀電子工程，後來在新訓中心擔任教育班長四年多，負責規劃訓練課程、管理團隊。退伍後我回到資工，花了一年半專心做後端專案。

我做過三個專案：第一個是演唱會搶票系統，主結構是我做的，我也自己抓出兩個併發問題並修好；第二個是四人團隊的電商後端，我擔任隊長；第三個是 Threat Intel Agent，我用 AI 協作的方式開發，負責實作 Claude 的 Tool Use 並部署上線。

我希望把後端和 AI 串接的經驗，用在關網把 AI 落地到電子發票和財稅服務的工作上。

2. 為什麼投遞關網（約 40 秒）

有三個原因。

第一，關網做電子發票，每個月處理近 3,000 萬張，不能出錯、資安要求高。我做搶票系統時最在意的就是大量請求下不能出錯，方向很接近。

第二，這個職缺是要把 Agent、RAG 用在真實業務上，不只是做展示，這是最想學的。

第三，這個職位要面對客戶。我當教育班長時每天跟不同背景的人溝通，我覺得自己適合把技術講給不懂技術的人聽。

3. 把我當客戶，解釋什麼是 AI（你的工廠比喻完整版）

AI 可以想成一座很特別的工廠。一般工廠只能放固定的原料，這座工廠可以放進各種東西，文字、聲音、圖片，甚至影片，它會先分析，再做出您要的成品。

例如您把一張憑證拍照放進去，它可以讀出統編、金額、日期；您問它「發票開錯了要作廢還是折讓」，它可以整理成一段說明。

不過這座工廠偶爾會出錯，而且出錯時看起來還是很有自信。所以我們會讓它只根據公司確認過的資料來做，重要的金額或送出動作，再由人確認一次。

簡單說，AI 負責把重複、耗時的事情做快，人負責最後把關。

被追問時對應的技術：各種原料 = 多模態；只根據確認過的資料 = RAG；出錯很有自信 = 幻覺；由人確認 = human-in-the-loop。

4. 其他外行人版本 (Java 職缺被問過 API)

名詞	一句話講給客戶聽
API	像餐廳點餐櫃台：照菜單點，廚房做好送出來，您不用知道廚房怎麼煮。系統之間也是照固定格式提要求、拿結果
RAG	像開書考試：先去翻公司的資料，再根據翻到的內容回答，不是憑印象
Agent	像會自己安排步驟的助理：您給目標，它自己決定先查什麼、用哪個工具，查完整理結果給您

5. 遇過最大的困難 (搶票系統，約 60 秒)

最困難的是搶票系統的併發問題，我遇到兩個。

第一個是資料庫沒有用連線池。FastAPI 是非同步的，但資料庫這段沒有配合，人一多請求就卡在資料庫，非同步的效果被抵銷。後來改成連線池重複使用連線。

第二個是鎖座位時，我用 Redis 的 SET NX，卻沒有檢查它回傳成功還是失敗，沒搶到鎖的人也繼續往下走。人少時看不出來，人一多就互搶。修法是只有真的拿到鎖才繼續，沒拿到就回「座位已被選走」。

修完之後，用 AI 幫我寫的壓測腳本模擬 10 到 600 人同時搶票，確認沒有超賣。

我學到兩件事：寫了 async 不代表會變快，整條路徑都不能卡住；呼叫了鎖不等於有鎖，一定要檢查結果。

細節被追問但你不確定時：

「這部分的實作細節我要回去對照程式碼才能講準，但原理是.....」

不要硬掰，說清楚原理比講錯細節好。

6. 程式是你寫的，還是 AI 寫的？

我會用 AI 協作開發，但每個專案分工不同。搶票系統的主結構是我做的，兩個併發問題也是我自己找出來、修好的，壓測腳本則是請 AI 寫的。Threat Intel Agent 的架構是 AI 協助規劃，我負責實作、串接和部署，也去弄懂每個設計的理由。

我覺得 AI 能讓開發變快，但工程師要能判斷它做得對不對，這是我一直在練的。

7. 衝突與意見不合（最常被追問，一定要有真實例子）

絕對不要 只回答「我沒有跟人起過衝突」。心得裡有人這樣答，被一路追問到被判定「沒有參與」。

今晚從教育班長或 MineMarket 隊長時期挑一件真實的事，填在這裡：

結構	【今晚填】
情境：什麼時候、跟誰	-----
分歧：大家的意見哪裡不一樣	-----
我做了什麼：先聽理由 → 回到共同目標 → 用什麼方式決定	-----
結果：最後怎麼做、效果如何	-----
學到什麼	-----

連續追問的接法：

追問	回答結構
「有沒有跟組員衝突？」	「正面衝突比較少，但意見不同一定有，例如〔上面那件事〕，我的做法是……」
「參與度給自己幾分？」	「我給自己〔X〕分。加分是我〔具體做了什麼〕；扣分是〔真實的不足〕，之後我會〔怎麼改〕。」
「你的幫忙有沒有成效？」	講結果：〔事情有沒有推進、有沒有再發生〕；如果效果有限，說清楚你學到什麼

8. 其他情境題（照這個順序講）

題目	回答順序
跟客戶起衝突	先讓客戶把話說完、處理情緒 → 確認他真正在意什麼（結果錯了，還是處理太慢）→ 說清楚做得好與做不到 → 給替代方案和回覆時間 → 事後記錄、同步團隊
想法被團隊集體否決	先問清楚否決原因，通常代表我沒考慮到的限制 → 值得保留的部分和大家的意見整合 → 如果最後不採用，就照團隊決定全力做
工作突發狀況	先確認影響範圍 → 能先止血就先止血（切回舊版、先人工處理）→ 馬上回報，不自己硬扛 → 找根本原因 → 記錄怎麼避免再發生
AI 答錯，客戶很生氣	先道歉 → 確認是哪一題、哪個答案 → 查是資料問題還是模型問題 → 告知處理時程 → 修正後回報
客戶要求 AI 百分之百正確	誠實說 AI 無法保證；說明怎麼降低錯誤：依據公司資料、附出處、重要動作由人確認

9. 關於你自己

題目	回答方向
優點	遇到問題會追到根本原因（搶票的兩個問題）；教育班長四年，習慣在壓力下帶人、溝通
缺點	實作和除錯的經驗比架構設計多，像 Agent 這種新領域，目前還會參考 AI 或文件來規劃。我的做法是不只做出來，還回頭弄懂每個設計為什麼這樣做
喜歡的主管	目標講清楚、會給回饋，包括直接指出我的錯
不喜歡的主管	講「做法」不批評「人」：目標常變又沒說明原因。遇到的話，我會主動確認優先順序、把討論結果記下來
求職最重視三個條件	能接觸真實的 AI 落地場景；有人可以討論、給回饋；團隊溝通透明
對工作的期待	前三個月熟悉系統和現有的 Agent、RAG 流程，能獨立做好測試與維護；之後能負責一個模組
為什麼從電子轉資工	【今晚填：真實原因，一兩句】
退伍後這一年半在做什麼	在明新資工讀書，同時專心做三個專案（說清楚不是在公司任職）
住竹北，通勤台北？	【今晚填：真實安排，例如搭什麼車、多久、有沒有搬家打算】
期望待遇	依公司規定；被追問就說尊重職缺範圍，希望依能力和表現評估
何時可上班	錄取後兩週（履歷寫的）

履歷上每個技術，都要能講一句正確的話

用法：遮住右邊兩欄，看左邊講出「它是什麼」和「我怎麼用」。卡住的才看答案。

技術	它是什麼 (一句正確說法)	你在專案裡怎麼用
FastAPI	Python 的非同步 Web 框架，用型別提示自動驗證參數，自動產生 API 文件 (/docs)	搶票系統、Threat Intel Agent 後端
Uvicorn	執行 FastAPI 的 ASGI 伺服器	同上
async / await	等待 I/O 時把執行權讓出去，讓一個程序同時服務很多請求；中間有阻塞呼叫就會卡住全部	搶票的連線池問題
連線池	預先建立一批資料庫連線重複使用，省去每次建立的成本，也限制同時連線數	搶票的修正
Race condition	多個請求同時讀寫同一份資料，結果取決於誰先誰後	搶票鎖票沒檢查結果
Redis	記憶體型 key-value 資料庫，速度快，常用在快取、鎖、計數、排隊	鎖座位、限購序列
SET NX	key 不存在時才設定成功，是單一原子操作；通常會加過期時間，避免程式當掉後鎖一直不放	鎖座位
MySQL	關聯式資料庫，用資料表和 SQL，支援交易 (一組操作全部成功或全部取消)	搶票、MineMarket
MongoDB	文件型 NoSQL 資料庫，存類似 JSON 的文件，欄位可以不固定	Agent 的報告與決策軌跡
Flask	輕量的 Python Web 框架，需要的功能靠擴充套件	MineMarket
Blueprint	Flask 把路由依功能拆成獨立模組	MineMarket 切四個模組，讓四個人平行開發
Jinja2	樣板引擎，伺服器端把資料填進 HTML	MineMarket 前端頁面
Decorator	包住一個函式，在前後加上共用邏輯，不用改原函式	MineMarket 共用 decorator；Agent 的快取與計時
inspect	Python 標準函式庫，可以讀取函式的參數資訊	decorator 自動組好參數
Session	登入狀態存在伺服器，瀏覽器只拿一個 session id (放在 cookie)	會員登入
TOTP	用共享密鑰加上當下時間算出一次性密碼，約 30 秒換一次 (Google Authenticator)	搶票購票前的雙重驗證
CheckMacValue	綠界用雙方金鑰對參數算出雜湊簽章，確認通知真的來自綠界、內容沒被改	MineMarket 金流
RCON	遠端對 Minecraft 伺服器下指令的協定	付款後即時發道具
Ngrok	幫本機服務開一個公開網址的通道	讓 Heroku 連到本機遊戲伺服器

技術	它是什麼 (一句正確說法)	你在專案裡怎麼用
Cloudinary	雲端圖片儲存服務	Heroku 重啟會清掉本機檔案，所以圖片放雲端
Heroku、Railway、Render	雲端部署平台，不用自己管伺服器	三個專案的部署
Docker	把程式和執行環境打包成映像檔，到哪裡都用同樣方式執行	Agent 部署到 Render
Tool Use	模型回傳想呼叫的工具和參數，程式執行後用 tool_result 回傳結果	Agent 核心流程
tool_choice	指定模型必須呼叫某個工具	強制呼叫 submit_report，交出固定格式報告
RESTful API	用 URL 表示資源、HTTP 方法表示動作	Agent 提供給防火牆查詢的 API
React、Vite	React 是元件化前端框架；Vite 是開發與打包工具	搶票前端【今晚確認：是你寫的還是隊友寫的】
Tailwind CSS	直接用 class 名稱套樣式的 CSS 框架	搶票、Agent 前端
Git、PR	版本控制；PR 是合併程式碼前讓人審查的請求	團隊協作
Django	功能齊全的 Python 框架，內建 ORM 與後台	履歷寫「接觸過」，不要說熟

原則 不會的就說「這個我沒有實際做過，我目前的理解是.....；如果要做，我會先.....」。不要硬掰，被追問露餡的傷害比說不會大。

關網資訊：通勤路上看這頁

項目	記住
是什麼	財政部核可的電子發票加值服務中心，幫企業開立、上傳、管理電子發票
成立	2014 年
規模	9,000 家以上企業客戶、每月近 3,000 萬張電子發票
資安	ISO 27001 (財政部要求加值中心必須通過)
產品 1	電子發票服務：雲端型、介接型、會員載具、境外公司服務
產品 2	VAT 雲端會計憑證辨識系統 (2023)：憑證辨識，主要給會計師、記帳士事務所
產品 3	快收慢付 (2024)：B2B 商務支付，跟遠東商銀 Bankee 合作
股東	國發基金、中光電智能雲服
股票	沒有上市櫃。「關貿網路 (6183)」是另一家同業，不要講錯
AI 部門	設在營運管理部底下，職位要面對客戶

「你有用過我們的產品嗎？」

我沒有企業帳號，所以沒有實際開過發票，但面試前我看過貴公司官網的電子發票查詢頁，也了解電子發票有雲端型、介接型這幾種方案，還有 VAT 憑證辨識和快收慢付。

出門前一定要真的打開官網看一次，這句話才是真的。

「你覺得 AI 可以用在我們哪裡？」

我想到三個方向：第一是憑證辨識，AI 讀出統編和金額，程式再檢核稅額，信心不足就轉人工；第二是法規和操作問題的客服，用公司公告和手冊做 RAG；第三是把內部系統包成工具給 Agent 使用。不過我了解團隊目前實際的優先順序是什麼。

最後反問 (挑 2 個)

1. 這個職位面對客戶的比例大概多少？主要是協助導入、需求訪談，還是技術支援？
2. 進來前三個月，最希望我先接手哪一塊？
3. 團隊目前怎麼評估 LLM 輸出的品質？

壓力、聲音、當天清單

被咄咄逼人時

情況	做法
語氣很衝、一直質疑	放慢語速，不辯解、不動氣。心得作者認為這是 抗壓測試 ，不是針對你
被指出不足	先承認：「您說得對，這部分我經驗確實不夠。」接著講你正在怎麼補
問到不會的	「我沒有實際做過，我的理解是.....」
筆試寫錯的題被問	「當時我選了 X，後來想想應該是 Y，因為.....」
腦袋空白	「我整理一下想法。」停兩秒再答，完全可以

聲音

時機	做法
今晚	練習時 小聲講 就好，不要用全音量反覆講
當天	喝 常溫或溫水 ；避免冰飲、濃咖啡；不要用力清喉嚨
面談中	平常音量，用 放慢和停頓換清楚 ，不要硬拉大聲；隨時可以喝水
要不要先說明	你自己決定。可以說：「我喉嚨之前受過傷，聲音比較沙，不影響溝通，沒聽清楚請直接跟我說。」不需要解釋病況

醫生若有給你用嗓的建議，以醫生為準。

今晚準備好

- ☐ 履歷紙本 1-2 份、筆
- ☐ 一瓶常溫水
- ☐ 乾淨整齊的衣服（心得沒寫服裝規定，整齊的襯衫或 POLO 即可）
- ☐ 查好路線：竹北到南京復興站附近，**留好緩衝時間**
- ☐ 手機充飽電、存好聯絡窗口電話
- ☐ PART 3 的【今晚填】都寫好了

出門前

- ☐ 打開關網官網的電子發票查詢頁看一次
- ☐ 小聲講一次自我介紹

到場

- ☐ 提早 10 分鐘到，喝水、手機靜音
- ☐ 筆試時手機收起來

最後一句：題目不難，你要做的是把真實經驗講清楚。搶票的兩個問題是你自己找到的，教育班長的四年是真的，AI 協作的分工你講得清楚。這三件事，就是你跟其他應徵者不一樣的地方。